JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Serializable

All Known Subinterfaces:

AsymmetricKey, Attribute, Attribute, Attributes, CertPathValidatorException.Reason, Connector.Argument, Connector.BooleanArgument, Connector.IntegerArgument, Connector.SelectedArgument, Connector.StringArgument, Control, Descriptor, DHPrivateKey, DHPublicKey, DocAttribute, DSAPrivateKey, DSAPublicKey, ECPrivateKey, ECPublicKey, EdECPrivateKey, EdECPublicKey, ExtendedRequest, ExtendedResponse, Externalizable, Key, Name, NotificationFilter, PBEKey, PrintJobAttribute, PrintRequestAttribute, PrintServiceAttribute, PrivateKey, PublicKey, QueryExp, RelationType, RemoteRef, RSAMultiPrimePrivateCrtKey, RSAPrivateCrtKey, RSAPrivateKey, RSAPublicKey, SecretKey, ServerRef, SupportedValuesAttribute, UnsolicitedNotification, ValueExp, XECPrivateKey, XECPublicKey

---

`public interface` **`Serializable`**

Serializability of a class is enabled by the class implementing the java.io.Serializable interface.

Warning: Deserialization of untrusted data is inherently dangerous and should be avoided. Untrusted data should be carefully validated according to the "Serialization and Deserialization" section of the Secure Coding Guidelines for Java SE. Serialization Filtering describes best practices for defensive use of serial filters.

Classes that do not implement this interface will not have any of their state serialized or deserialized. All subtypes of a serializable class are themselves serializable. The serialization interface has no methods or fields and serves only to identify the semantics of being serializable.

It is possible for subtypes of non-serializable classes to be serialized and deserialized. During serialization, no data will be written for the fields of non-serializable superclasses. During deserialization, the fields of non-serializable superclasses will be initialized using the no-arg constructor of the first (bottommost) non-serializable superclass. This constructor must be accessible to the subclass that is being deserialized. It is an error to declare a class Serializable if this is not the case; the error will be detected at runtime. A serializable subtype may assume responsibility for saving and restoring the state of a non-serializable supertype's public, protected, and (if accessible) package-access fields. See the *Java Object Serialization Specification,* section 3.1, for a detailed specification of the deserialization process, including handling of serializable and non-serializable classes.

When traversing a graph, an object may be encountered that does not support the Serializable interface. In this case the NotSerializableException will be thrown and will identify the class of the non-serializable object.

Classes that require special handling during the serialization and deserialization process must implement special methods with these exact signatures:

```
private void writeObject(java.io.ObjectOutputStream out)
    throws IOException;
private void readObject(java.io.ObjectInputStream in)
    throws IOException, ClassNotFoundException;
private void readObjectNoData()
    throws ObjectStreamException;
```

The writeObject method is responsible for writing the state of the object for its particular class so that the corresponding readObject method can restore it. The default mechanism for saving the Object's fields can be invoked by calling out.defaultWriteObject. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is saved by writing the individual fields to the ObjectOutputStream using the writeObject method or by using the methods for primitive data types supported by DataOutput.

The readObject method is responsible for reading from the stream and restoring the classes fields. It may call in.defaultReadObject to invoke the default mechanism for restoring the object's non-static and non-transient fields. The defaultReadObject method uses information in the stream to assign the fields of the object saved in the stream with the correspondingly named fields in the current object. This handles the case when the class has evolved to add new fields. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is restored by reading data from the ObjectInputStream for the individual fields and making assignments to the appropriate fields of the object. Reading primitive data types is supported by DataInput.

The readObjectNoData method is responsible for initializing the state of the object for its particular class in the event that the serialization stream does not list the given class as a superclass of the object being deserialized. This may occur in cases where the receiving party uses a different version of the deserialized instance's class than the sending party, and the receiver's version extends classes that are not extended by the sender's version. This may also occur if the serialization stream has been tampered; hence, readObjectNoData is useful for initializing deserialized objects properly despite a "hostile" or incomplete source stream.

Serializable classes that need to designate an alternative object to be used when writing an object to the stream should implement this special method with the exact signature:

```
ANY-ACCESS-MODIFIER Object writeReplace() throws ObjectStreamException;
```

This writeReplace method is invoked by serialization if the method exists and it would be accessible from a method defined within the class of the object being serialized. Thus, the method can have private, protected and package-private access. Subclass access to this method follows java accessibility rules.

Classes that need to designate a replacement when an instance of it is read from the stream should implement this special method with the exact signature.

```
ANY-ACCESS-MODIFIER Object readResolve() throws ObjectStreamException;
```

This readResolve method follows the same invocation rules and accessibility rules as writeReplace.

Enum types are all serializable and receive treatment defined by the *Java Object Serialization Specification* during serialization and deserialization. Any declarations of the special handling methods discussed above are ignored for enum types.

Record classes can implement `Serializable` and receive treatment defined by the *Java Object Serialization Specification,* Section 1.13, "Serialization of Records". Any declarations of the special handling methods discussed above are ignored for record types.

The serialization runtime associates with each serializable class a version number, called a serialVersionUID, which is used during deserialization to verify that the sender and receiver of a serialized object have loaded classes for that object that are compatible with respect to serialization. If the receiver has loaded a class for the object that has a different serialVersionUID than that of the corresponding sender's class, then deserialization will result in an `InvalidClassException`. A serializable class can declare its own serialVersionUID explicitly by declaring a field named `"serialVersionUID"` that must be static, final, and of type `long`:

```
ANY-ACCESS-MODIFIER static final long serialVersionUID = 42L;
```

If a serializable class does not explicitly declare a serialVersionUID, then the serialization runtime will calculate a default serialVersionUID value for that class based on various aspects of the class, as described in the *Java Object*

Serialization Specification. This specification defines the serialVersionUID of an enum type to be 0L. However, it is *strongly recommended* that all serializable classes other than enum types explicitly declare serialVersionUID values, since the default serialVersionUID computation is highly sensitive to class details that may vary depending on compiler implementations, and can thus result in unexpected `InvalidClassExceptions` during deserialization. Therefore, to guarantee a consistent serialVersionUID value across different java compiler implementations, a serializable class must declare an explicit serialVersionUID value. It is also strongly advised that explicit serialVersionUID declarations use the `private` modifier where possible, since such declarations apply only to the immediately declaring class--serialVersionUID fields are not useful as inherited members. Array classes cannot declare an explicit serialVersionUID, so they always have the default computed value, but the requirement for matching serialVersionUID values is waived for array classes.

Since:

1.1

External Specifications

Java Object Serialization Specification

See Also:

`ObjectOutputStream`, `ObjectInputStream`, `ObjectOutput`, `ObjectInput`, `Externalizable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.